

SESIÓN 04

Modelos, rigging y animación del personaje

Del modelo 3D al personaje controlable en Unity



Modelo 3D

Importación y preparación del modelo



Rigging

Estructura de huesos y configuración



Animación

Clips, Animator y transiciones



Programación

Control del personaje e implementación de salto



Logro de la sesión

El estudiante implementa modelos, rigging y animaciones del personaje en Unity y programa el salto, integrando de manera coherente estos elementos en su prototipo de videojuego.



Tiempo de la sesión

35 min teoría

45 min práctica guiada

10 min cierre



Desarrollo de Videojuegos



Caso: Guardianes del Agua



Unidad I: Diseño de videojuegos



1. Diseñamos la experiencia

Planificamos la historia, el personaje, el objetivo, el conflicto, el escenario y las mecánicas.



2. Construimos el escenario

Creamos y organizamos los GameObjects, aplicamos materiales, iluminación y configuramos la cámara.



3. Movimos al personaje

Implementamos el desplazamiento básico del Player con scripts en C# y entrada del usuario.



4. Empezamos a interactuar

Usamos Colliders y Trigger para detectar objetos y ejecutar acciones cuando el jugador interactúa.



De lo estático a lo interactivo

En la semana pasada dimos el paso de un escenario visual a un mundo que responde al jugador. Ahora llevaremos a nuestro personaje al siguiente nivel con modelos, rigging, animaciones y salto.



Pregunta de inicio

Si nuestro personaje ya puede moverse e interactuar con objetos, ¿cómo podemos hacerlo más realista, expresivo y capaz de saltar obstáculos?





Al finalizar la sesión

El estudiante implementa **modelos, rigging y animaciones** del personaje en Unity y programa el **salto**, integrando de manera coherente estos elementos en su prototipo de videojuego.



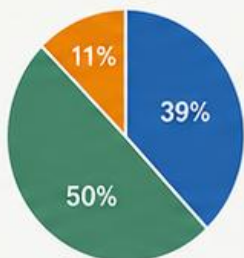
Evidencia

Escena funcional en Unity donde el personaje cuenta con modelo 3D, rigging, animaciones básicas (**Idle, Walk y Jump**) y puede ejecutar el salto correctamente.



Tiempo

- **35 min** teoría
- **45 min** práctica guiada
- **10 min** cierre



- Teoría (35 min)
- Práctica (45 min)
- Cierre (10 min)

Modelo 3D → Rigging → Animación → Personaje en juego



Pregunta de inicio

Si nuestro personaje ya puede moverse, ¿cómo hacemos para que tenga una estructura adecuada, se anime de forma realista y pueda saltar como en un juego profesional?





Idea clave

Un **GameObject** por sí solo representa un elemento dentro de la escena. Los **Components** agregan características: apariencia, colisión, audio, física o comportamiento. Un **script** permite definir reglas y respuestas mediante código.



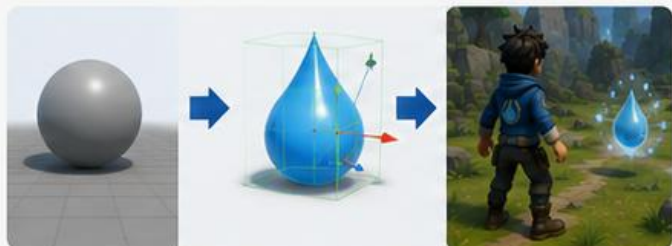
Cuando hay interacción

Cuando el jugador realiza una acción y el sistema responde, comenzamos a construir interacción en el videojuego.



Ejemplo

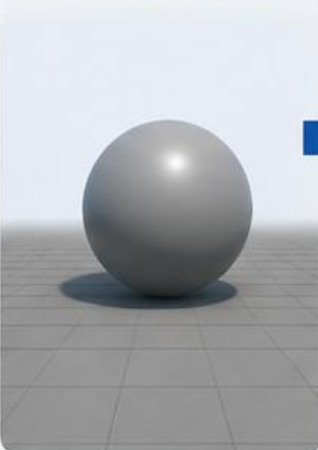
Una **Sphere** puede representar una gota de agua. Con un **Collider** y un **script** puede convertirse en un objeto que el jugador detecta y recolecta.



Transformación de un objeto en la escena

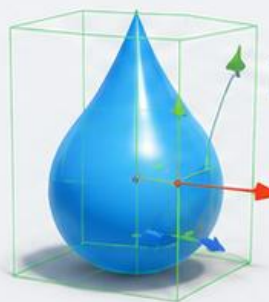
1. GameObject básico

Solo representa un objeto en la escena.



2. + Components

Se agregan características como **Collider** y **Render**.



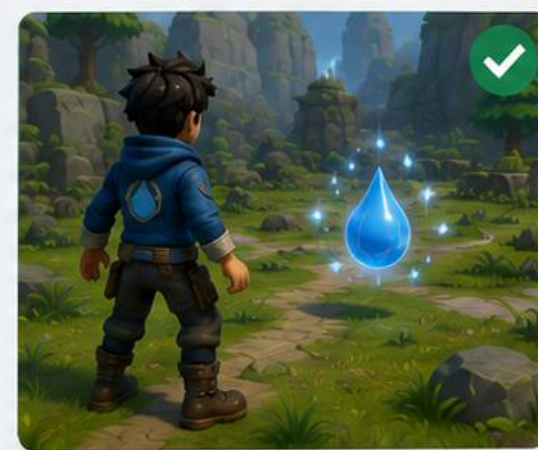
3. + Script

Se asocia un script que define su comportamiento.

```
C#
1 public class Gota : MonoBehaviour
2 {
3     public int valor = 10;
4
5     void OnTriggerEnter(Collider other)
6     {
7         if (other.CompareTag("Player"))
8         {
9             Recolectar();
10        }
11    }
12
13    void Recolectar()
14    {
15        // Lógica de recolección
16        Destroy(gameObject);
17    }
18 }
```

4. Objeto interactivo

El jugador puede detectarla y recolectarla.



Componentes comunes utilizados



Transform

Posición, rotación y escala del objeto.



Mesh Renderer

Muestra el modelo o material.



Collider

Detecta colisiones e interacciones.



Script

Define el comportamiento.



Audio Source

Reproduce sonidos y efectos.



En resumen

Un **GameObject** con los componentes adecuados y un script asociado puede responder a las acciones del jugador y a los eventos del juego.



Aplicación a Guardianes del Agua

Las gotas solo serán relevantes cuando el jugador pueda detectarlas y recolectarlas.





Definición

Un script es un **componente programable** que define el **comportamiento** de un GameObject.

En Unity se utiliza principalmente **C#** para crear scripts.

Un mismo script puede controlar movimiento, interacción, puntuación, estados, eventos y otras reglas.



Idea clave

GameObject + Components + Script
= **objeto con características y comportamiento.**



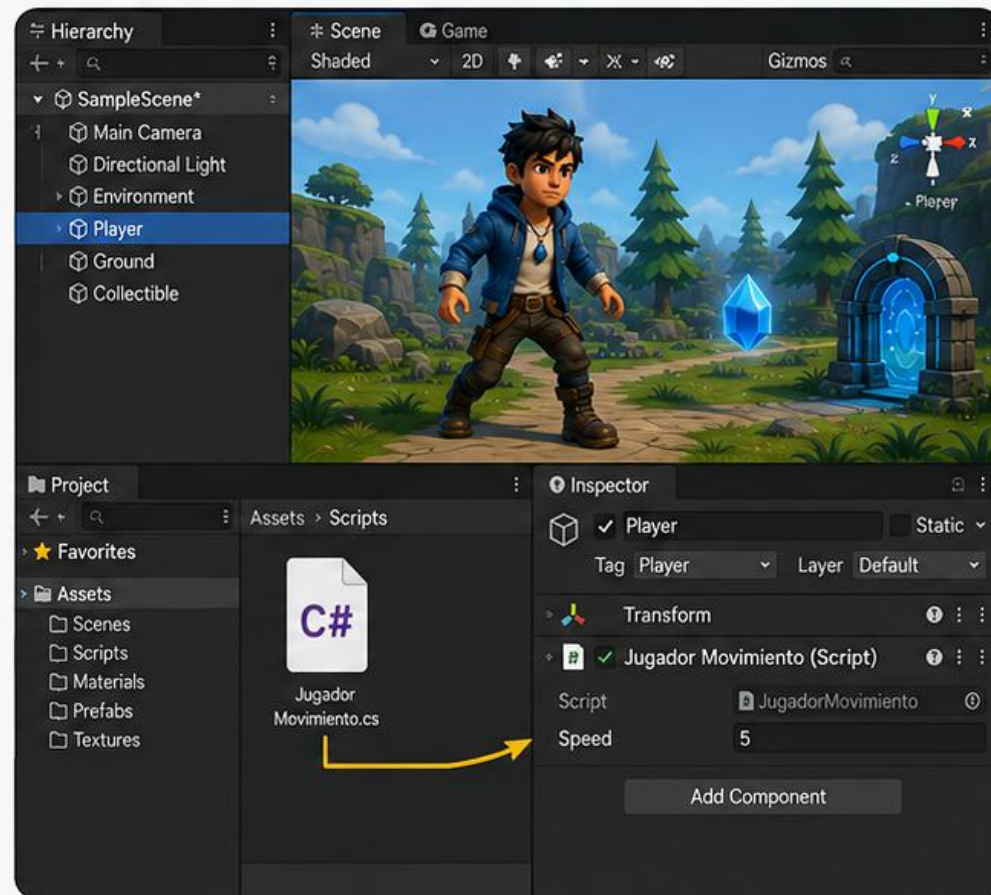
Cómo funciona

- 1 El script se crea como un archivo del proyecto.
- 2 Se asocia al GameObject que debe ejecutar ese comportamiento.
- 3 Unity ejecuta el script cuando ocurren eventos (Start, Update, colisiones, entrada del usuario, etc.).

Asociación del script al GameObject



Ejemplo: Player con script de movimiento



En resumen

Un script convierte un objeto estático en un objeto inteligente que puede moverse, detectar, recolectar, puntuar, cambiar de estado y mucho más.



Aplicación a Guardianes del Agua

El Player necesita un script para moverse, detectar gotas de agua, sumar puntos y activar la fuente cuando complete su misión.





¿Qué es un componente?

Los componentes añaden funcionalidades a un **GameObject**. Un mismo **GameObject** puede tener varios componentes al mismo tiempo.



Idea clave

Los componentes permiten construir comportamientos complejos combinando características simples.

Componentes más utilizados



Transform

Define la posición, rotación y escala del objeto.



Collider

Detecta colisiones con otros objetos.



Rigidbody

Aplica físicas como gravedad, fuerzas e impulsos.



Animator

Controla las animaciones a través de un controlador de animaciones.



Script

Permite programar comportamientos personalizados en C#.



Audio Source

Reproduce sonidos y efectos de audio.

Ejemplo: Player con varios componentes



Flujo de trabajo



Crear **GameObject**
(ej. Player)



Añadir componentes
necesarios



Configurar
propiedades



Probar en el juego
y ajustar



En resumen

Los componentes son la base para construir el comportamiento



Aplicación en Guardianes del Agua

El Player necesita varios componentes para moverse, detectar colisiones,



¿Qué es el rigging?

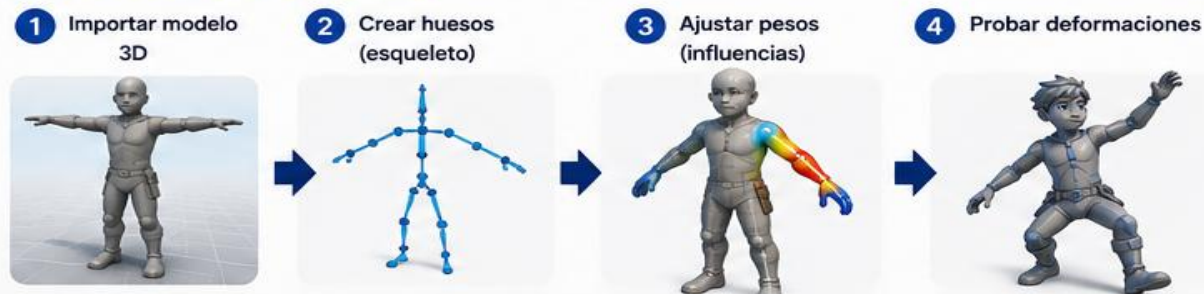
El rigging es el proceso de crear un esqueleto (huesos) dentro del modelo 3D para poder deformarlo y animarlo. Permite que el personaje se mueva de forma natural.



Idea clave

Un buen rig permite animaciones realistas y controlables. Define cómo se moverá cada parte del personaje.

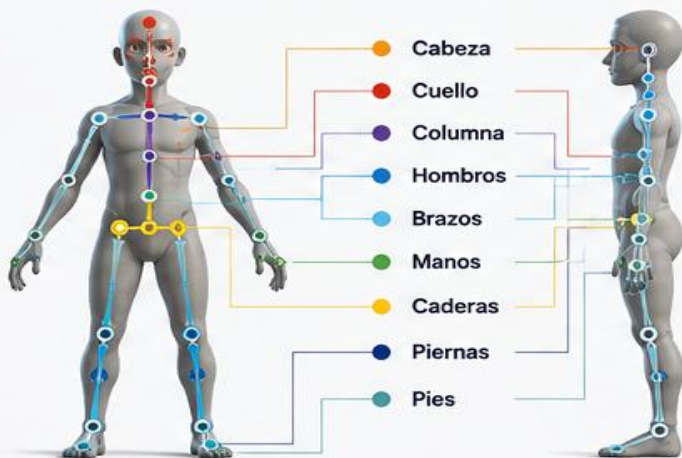
Proceso de rigging en Unity



Estructura básica de un esqueleto humano

Vista frontal

Vista lateral



Herramientas de rigging en Unity



Bone Tool

Permite crear y posicionar los huesos que formarán el esqueleto.



Configure

Asigna la jerarquía padre-hijo entre los huesos.



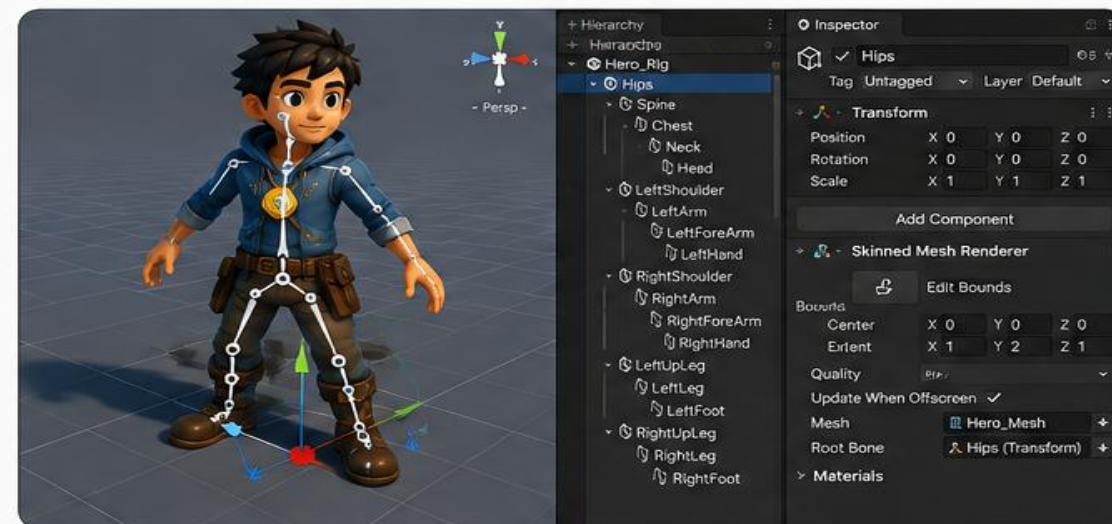
Skin Weights

Define la influencia de cada hueso sobre la malla del modelo.



Pose

Prueba y ajusta las deformaciones del personaje.



En resumen

El rigging convierte un modelo 3D estático en un personaje articulado. Crear un esqueleto correcto y asignar bien los pesos permite que las animaciones se vean naturales y sin deformaciones extrañas.



Aplicación en Guardianes del Agua

El esqueleto del personaje del Guardián permite animarlo para caminar, correr, saltar y realizar acciones como recolectar gotas de agua y abrir portales mágicos.





¿Qué es la animación?

La animación es el proceso de crear movimientos para el personaje a lo largo del tiempo. En Unity usamos clips de animación y un controlador para reproducirlas de manera fluida y coherente.



Idea clave

Una buena animación hace que el personaje se vea creíble, responda al jugador y transmita emociones.

Flujo de animación en Unity



1. Clips de animación

Movimientos individuales



2. Animator Controller

Gestiona y combina clips



3. Estados y transiciones

Reacciona a condiciones



4. Entrada del jugador

Controles y parámetros

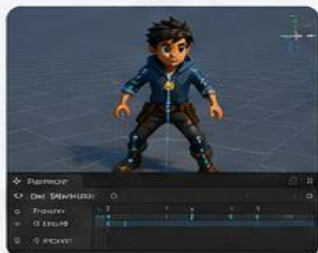


5. Animación en juego

Movimiento final en escena

Tipos de animación

Animación por claves (Keyframe)



Cada posición se define manualmente en el tiempo.

Animación por captura de movimiento (Mocap)



Se capturan movimientos reales y se aplican al modelo 3D.

Animación procedural



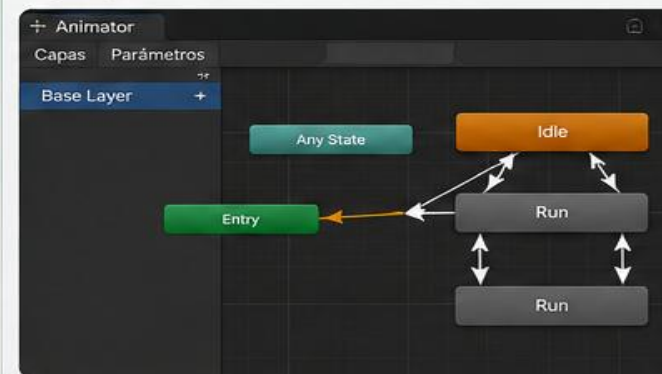
El movimiento se genera mediante algoritmos y reglas.

Animación híbrida



Combina claves, mocap y procedural para mejores resultados.

Ejemplo: Animator Controller



Parámetros

Speed	0.5
Jump	☒
Grounded	☒
Attack	☐

Los parámetros controlan las transiciones entre estados según la entrada del jugador y la lógica del juego.

Clips de animación comunes



Idle
(Reposo)

Walk
(Caminar)

Run
(Correr)

Jump
(Saltar)

Attack
(Atacar)

Die
(Morir)



Aplicación en Guardianes del Agua

El Guardián utiliza distintas animaciones para moverse, saltar, atacar y reaccionar a los eventos del juego.

La combinación de clips y transiciones permite que el personaje se sienta vivo y responda a las acciones del jugador.



En resumen

La animación da vida al personaje y es esencial para una buena experiencia de juego. En Unity usamos clips de animación, un Animator Controller y parámetros para crear movimientos fluidos y naturales que responden al jugador y al mundo del juego.



Recuerda: una buena animación comunica intención, emoción y mejora la inmersión.



¿Por qué usar parámetros?

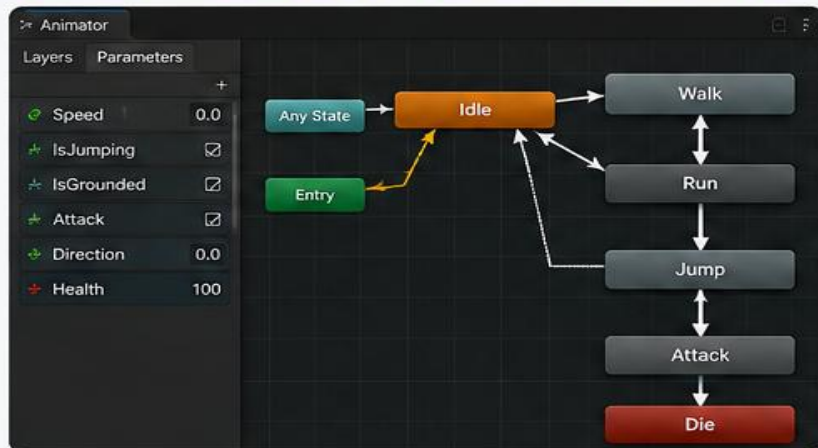
Los parámetros permiten que las animaciones cambien según las acciones del jugador o las condiciones del juego, haciendo al personaje más inteligente y realista.



Idea clave

Un Animator Controller usa parámetros y transiciones para decidir qué animación reproducir y cuándo cambiar a otra.

Ventana Animator: Estados, transiciones y parámetros



- Estados**
Cada bloque representa una animación.
- Transiciones**
Las flechas indican el flujo entre estados.
- Parámetros**
Controlan las condiciones de las transiciones.
- Condiciones**
Definen cuándo ocurre el cambio (true/false, mayor que, etc.).

Flujo de trabajo con parámetros



Ejemplo: Transiciones comunes

Transición	Condición (ejemplo)	Descripción
Idle → Walk	Speed > 0.1	Cuando el personaje empieza a moverse.
Walk → Run	Speed > 3	Cuando la velocidad aumenta.
Run → Idle	Speed < 0.1	Cuando el personaje se detiene.
Any State → Jump	IsJumping = true	Cuando el jugador presiona saltar.
Any State → Attack	Attack = true	Cuando el jugador ataca.
Any State → Die	Health <= 0	Cuando la vida llega a cero.



Las transiciones pueden tener duración (Transition Duration) para que el cambio entre animaciones sea más suave.

DURACIÓN

Control de parámetros desde código (C#)

```

public class PlayerController : MonoBehaviour
{
    public Animator animator;
    public float speed;

    void Update()
    {
        // Ejemplo: mover personaje
        speed = Input.GetAxis("Horizontal");
        animator.SetFloat("Speed", Mathf.Abs(speed));

        if (Input.GetButtonDown("Jump"))
            animator.SetBool("IsJumping", true);

        if (Input.GetButtonDown("Fire1"))
            animator.SetTrigger("Attack");
    }
}

```

Nota importante

- ✓ Usa SetFloat() para valores decimales (velocidad, dirección).
- ✓ Usa SetBool() para valores verdadero/falso (saltar, atacando).
- ✓ Usa SetTrigger() para acciones que ocurren una sola vez.
- ✓ Define parámetros con nombres claros y consistentes.



Aplicación en Guardianes del Agua

El personaje cambia de animación según su estado y las acciones del jugador.



Gracias a los parámetros y transiciones, el Guardián responde de forma natural en cada situación del juego.





¿Qué es una máquina de estados?

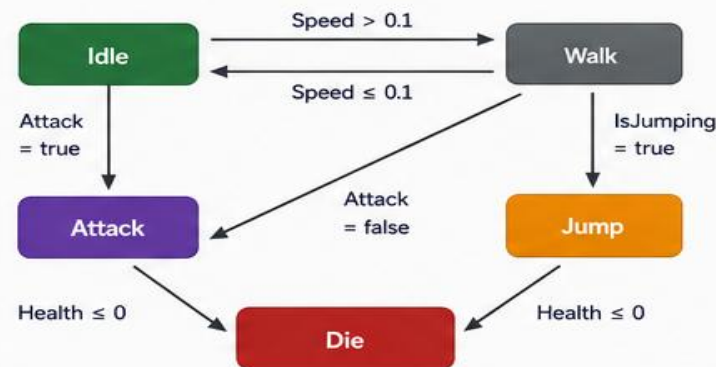
Una máquina de estados organiza las animaciones en estados y transiciones. Permite controlar comportamientos complejos de manera ordenada y reutilizable.



Idea clave

Las máquinas de estados permiten crear comportamientos realistas definiendo reglas claras para pasar de un estado a otro.

Ejemplo: Máquina de estados para un personaje



- **Idle:** El personaje está quieto.
- **Walk:** El personaje camina.
- **Attack:** El personaje ataca.
- **Jump:** El personaje salta.
- **Die:** El personaje muere.

i Cada transición ocurre cuando se cumple una condición definida.

Componentes clave



Estados

Representan una animación específica.
Ej.: Idle, Walk, Attack.



Transiciones

Conectan estados y definen las condiciones para cambiar de uno a otro.



Parámetros

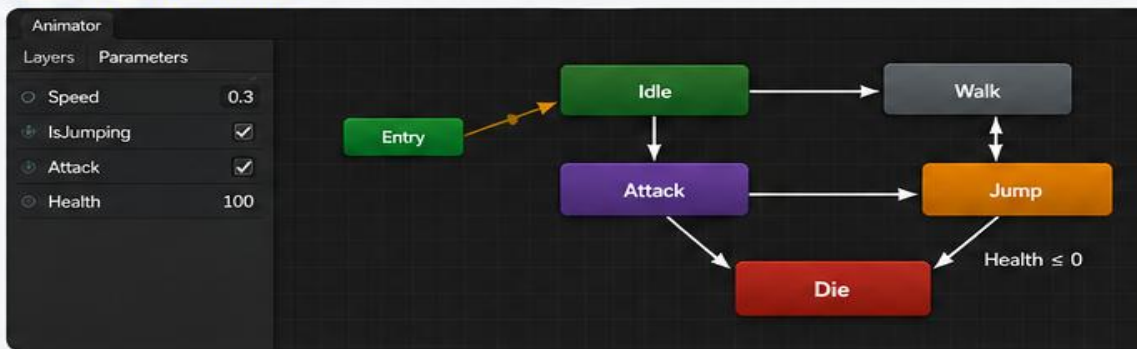
Variables que se usan en las condiciones.
Ej.: Speed (float), IsJumping (bool), Health (int).



Condiciones

Reglas que determinan cuándo ocurre la transición.
Ej.: Speed > 0.1, Health ≤ 0.

Vista en Unity Animator



Consejos prácticos

- ✓ Usa parámetros con nombres claros y consistentes.
- ✓ Evita transiciones innecesarias o contradictorias.
- ✓ Prueba cada condición en tiempo real.
- ✓ Ajusta las duraciones de transición para que sea más fluido.

Ejemplo de condiciones comunes

Condición	Tipo	Descripción
Speed > 0.1	Float	Si la velocidad es mayor a 0.1, caminar.
IsJumping = true	Bool	Si está saltando, ir al estado Jump.
Attack = true	Bool	Si ataca, ir al estado Attack.
Health ≤ 0	Int	Si la vida es 0 o menos, ir al estado Die.



Aplicación en Guardianes del Agua

El Guardián usa una máquina de estados para decidir su comportamiento según las acciones del jugador y su estado actual.





¿Qué es un evento de animación?

Los eventos de animación permiten ejecutar funciones o acciones en momentos específicos de un clip de animación, como reproducir sonidos, crear efectos o aplicar daño.



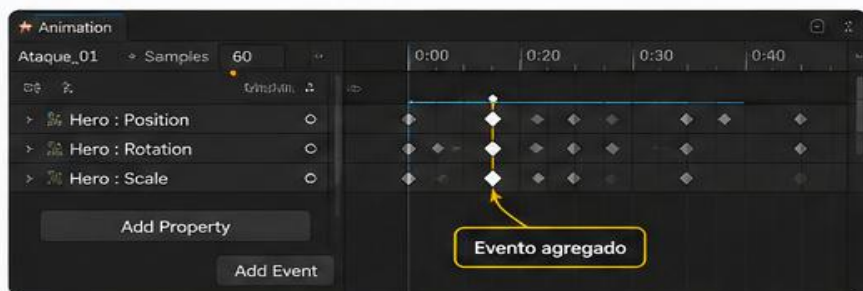
Idea clave

Usa eventos para sincronizar la lógica del juego con la animación de manera precisa y profesional.

¿Cómo funcionan los eventos de animación?



Ventana Animation: eventos



El rombo blanco en la línea de tiempo indica un evento en ese instante. Se puede arrastrar para ajustar el momento exacto.

Ejemplo práctico: ataque con sonido y efecto



Funciones comunes para eventos

Función	Descripción
ReproducirSonido()	Reproduce un clip de sonido.
CrearEfecto()	Instancia un efecto de partículas.
AplicarDaño()	Aplica daño al enemigo.
ActivarHitbox()	Activa o desactiva el hitbox.
PasoSonido()	Reproduce sonido de paso.
CámaraSacudida()	Genera una pequeña sacudida en la cámara.

Código de ejemplo (C#)

```
public class EventosAtaque : MonoBehaviour
{
    public AudioClip sonidoImpacto;
    public GameObject efectoImpacto;
    public int daño = 25;

    // Estas funciones se llaman desde los eventos de animación
    public void ReproducirSonidoImpacto()
    {
        AudioSource.PlayClipAtPoint(sonidoImpacto, transform.position);
    }

    public void CrearEfectoImpacto()
    {
        Instantiate(efectoImpacto, transform.position, Quaternion.identity);
    }

    public void AplicarDaño()
    {
        // Lógica para detectar enemigo y aplicar daño
        Debug.Log("Daño aplicado: " + daño);
    }
}
```

★ Nota importante

- ✓ Los eventos solo funcionan cuando el clip se reproduce.
- ✓ Asegúrate de que el objeto tenga el componente con las funciones asignadas.
- ✓ Puedes agregar varios eventos en un mismo clip.
- ✓ Úsalo para sincronizar sonido, efectos y lógica de juego.



Aplicación en Guardianes del Agua

El Guardián utiliza eventos para coordinar sus ataques y habilidades:

Ataque básico



Golpe especial



Salto



Habilidad de agua





¿Qué es un evento de animación?

Un evento permite ejecutar una función en un momento específico de un clip de animación, como reproducir un sonido, crear un efecto o aplicar daño.



Idea clave

Los eventos sincronizan la lógica del juego con la animación para crear experiencias más realistas e inmersivas.

¿Cómo funcionan los eventos de animación?

1 Selecciona el clip de animación



Abre la ventana Animation y elige el clip.

2 Agrega un evento en la línea de tiempo



Haz clic en "Add Event" en el momento deseado.

3 Asigna una función a ejecutar



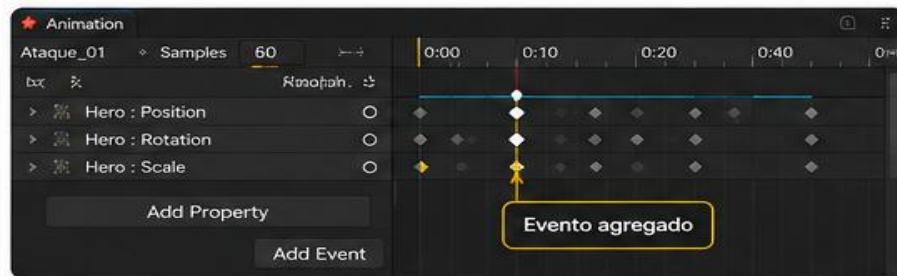
Elige la función del script que se ejecutará cuando ocurra el evento.

4 Ejecuta la animación en el juego



El evento se activa en el momento exacto definido en el clip.

Ventana Animation: eventos

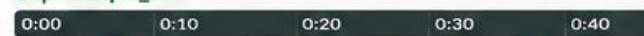


El rombo blanco en la línea de tiempo indica un evento en ese instante. Puedes arrastrarlo para ajustar el momento exacto.

Ejemplo práctico: ataque con sonido y efecto



Clip: Ataque_01



Crear efecto de espada
0:08

Reproducir sonido de impacto
0:18

Aplicar daño al enemigo
0:28

Funciones comunes para eventos

Función	Descripción
ReproducirSonido()	Reproduce un clip de sonido.
CrearEfecto()	Instancia un efecto de partículas.
AplicarDaño()	Aplica daño al enemigo.
ActivarHitbox()	Activa o desactiva el hitbox.
PasoSonido()	Reproduce sonido de paso.
CámaraSacudida()	Genera una pequeña sacudida en la cámara.

Código de ejemplo (C#)

```
public class EventosAtaque : MonoBehaviour
{
    public AudioClip sonidoImpacto;
    public GameObject efectoImpacto;
    public int dano = 25;

    // Evento: reproducir sonido de impacto
    public void ReproducirSonidoImpacto()
    {
        AudioSource.PlayClipAtPoint(sonidoImpacto, transform.position);
    }

    // Evento: crear efecto de espada
    public void CrearEfectoEspada()
    {
        Instantiate(efectoImpacto, transform.position, Quaternion.identity);
    }

    // Evento: aplicar daño al enemigo
    public void AplicarDañoEnemigo()
    {
        Enemigo enemigo = FindObjectOfType<Enemigo>();
        if (enemigo != null)
        {
            enemigo.RecibirDaño(dano);
        }
    }
}
```

★ Notas importantes

- ✓ Los eventos solo funcionan cuando el clip se reproduce.
- ✓ Asegúrate de que el objeto tenga el componente con las funciones asignadas.
- ✓ Puedes agregar varios eventos en un mismo clip.
- ✓ Prueba los eventos en tiempo real para verificar la sincronía.



Aplicación en Guardianes del Agua

Los eventos permiten que los ataques del Guardián sean impactantes y bien sincronizados.

1. Crear efecto de espada



En el inicio del ataque se crea el efecto visual.

2. Reproducir sonido



En el momento del impacto se reproduce el sonido.

3. Aplicar daño



El enemigo recibe daño en el instante exacto.

4. Desactivar hitbox



Al finalizar el ataque se desactiva el hitbox para evitar errores.



En resumen

Los eventos de animación permiten ejecutar acciones precisas dentro de un clip, como sonidos, efectos y daño. Usarlos correctamente mejora la calidad y realismo del juego.



Recuerda: Sincronizar la animación con la lógica del juego es clave para una experiencia profesional.





¿Qué es un clip de animación?

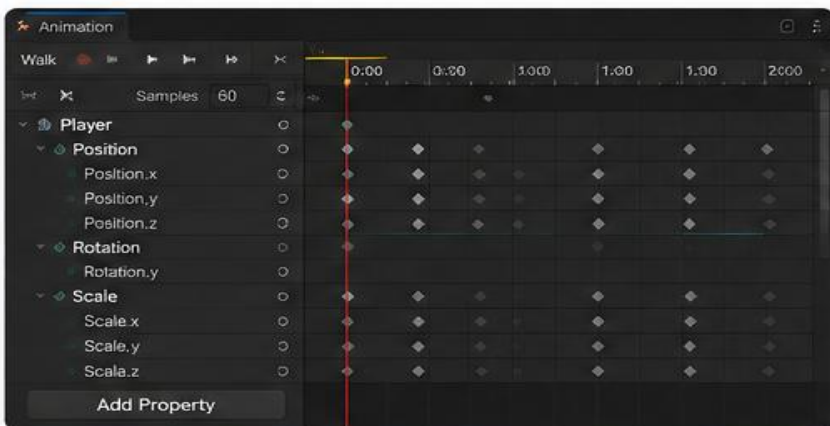
Un clip de animación es un conjunto de keyframes que definen los cambios de propiedades de un objeto en el tiempo. Se puede reutilizar en diferentes estados y personajes.



Idea clave

Utiliza clips de animación para crear movimientos consistentes y reutilizables, ahorrando tiempo y recursos.

Ventana Animation: keyframes



Los keyframes se interpolan automáticamente para crear movimientos suaves.



Keyframe

Marca un cambio de valor en un momento determinado.



Tiempo

Representa la duración total del clip.



Propiedades

Atributos que puedes animar (posición, rotación, escala, etc.).

Flujo para crear un clip de animación

1 Seleccionar objeto



Selecciona el objeto que quieres animar en la escena.



2 Crear clip



Crea un nuevo clip desde la ventana Animation.



3 Grabar keyframes



Modifica propiedades en el tiempo y el sistema registra keyframes.



4 Guardar clip



Guarda el clip en la carpeta del proyecto.



5 Usar en Animator



Asigna el clip a un estado en la ventana Animator.

Ejemplos de clips de animación comunes

Clip	Descripción	Uso típico
Idle	El personaje está en reposo.	Estado por defecto cuando no hay entrada.
Walk	El personaje camina a velocidad normal.	Movimiento básico de desplazamiento.
Run	El personaje corre más rápido.	Escapar, perseguir o desplazamiento rápido.
Jump	El personaje realiza un salto.	Superar obstáculos, acción de plataformas.
Attack	El personaje ejecuta un ataque.	Combatir enemigos.
Die	El personaje muere o cae derrotado.	Cuando la vida llega a cero.

Asignar un clip desde código (C#)

```
using UnityEngine;
using UnityEngine.Animations;

public class AsignarAnimacion : MonoBehaviour
{
    public Animator animator;
    public AnimationClip clipCorrer;

    void Start()
    {
        // Asignamos el clip al estado "Run"
        animator.runtimeAnimatorController
            .animationClips[0] = clipCorrer;
    }
}
```



Nota importante



Nombra tus clips de forma clara y consistente.



Usa clips reutilizables para mantener tu proyecto organizado.



Ajusta la duración y la curva de interpolación para lograr movimientos naturales.



Aplicación en Guardianes del Agua

El Guardián utiliza diferentes clips de animación para moverse, atacar y reaccionar.

Idle



Walk



Run



Jump



Attack



Die



Cada clip se reutiliza en diferentes situaciones para lograr una experiencia fluida y coherente.





¿Qué son los parámetros?

Los parámetros son variables que el Animator usa para evaluar las condiciones de las transiciones. Permiten crear animaciones inteligentes que responden a las acciones y al estado del juego.



Idea clave

Usa los parámetros correctos y combina condiciones para crear comportamientos más realistas y dinámicos.

Tipos de parámetros en Animator

Tipo	Descripción	Ejemplo de uso
Float	Valor decimal (puede ser negativo).	Speed, Horizontal
Int	Número entero.	Combo, Ammo
Bool	Verdadero o falso.	IsJumping, IsGrounded
Trigger	Activado una sola vez y se reinicia.	Attack, Shoot

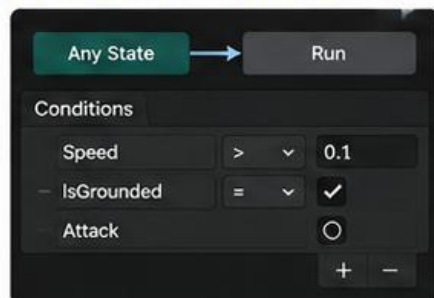


Elige el tipo de parámetro según el dato que necesitas controlar.

Ventana Inspector: parámetros

Parameters	
Speed	0.0
Horizontal	0.0
IsGrounded	☒
IsJumping	☐
Combo	0
Attack	☐

Condiciones en las transiciones



- ☒ **Speed > 0.1**
El personaje se mueve.
- ☒ **IsGrounded = true**
El personaje está en el suelo.
- ☒ **Attack (Trigger)**
Se activó el ataque.

i Puedes combinar varias condiciones usando AND (todas deben cumplirse).

Ejemplo: controlar parámetros desde código (C#)

```
public class PlayerController : MonoBehaviour
{
    Animator animator;

    void Update()
    {
        float h = Input.GetAxis("Horizontal");
        float v = Input.GetAxis("Vertical");

        animator.SetFloat("Horizontal", h);
        animator.SetFloat("Speed", v);
        animator.SetBool("IsGrounded", isGrounded);

        if (Input.GetButtonDown("Fire1"))
            animator.SetTrigger("Attack");
    }
}
```

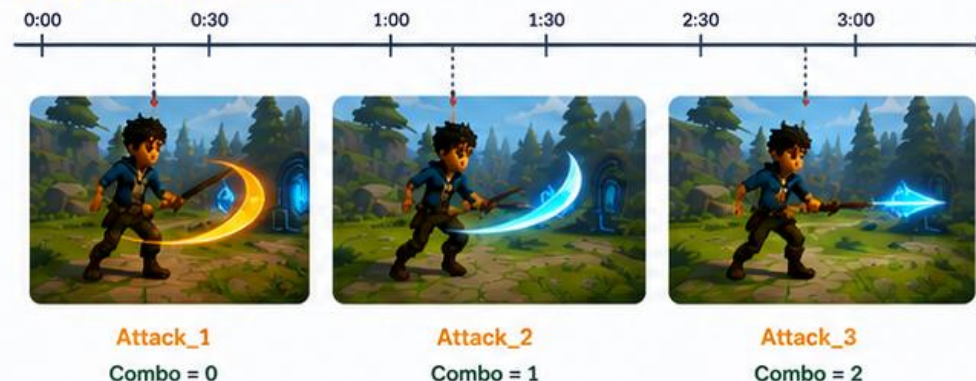
★ Nota importante

- ☒ Evita demasiados parámetros innecesarios.
- ☒ Nombra los parámetros de forma clara y consistente.
- ☒ Usa triggers para acciones instantáneas (ataques, disparos, etc.).
- ☒ Prueba cada transición para asegurar un comportamiento correcto.

Ejemplo: Combo de ataque con parámetros



Línea de tiempo del combo



Usa triggers para iniciar acciones y enteros para encadenar combos o secuencias. Esto permite crear animaciones fluidas y responsivas.



Buenas prácticas

- ☒ Mantén tu máquina de estados organizada y legible.
- ☒ Usa nombres descriptivos para parámetros y estados.
- ☒ Combina condiciones de forma lógica.
- ☒ Prueba en Play Mode constantemente.
- ☒ Evita transiciones muy rápidas que puedan causar cortes.



Tip pro

Usa capas (Layers) para separar acciones superiores (ataques) del movimiento básico. Esto permite más control y flexibilidad.





¿Qué son las capas?

Las capas permiten superponer animaciones para controlar diferentes partes del cuerpo o comportamientos al mismo tiempo. Son útiles para separar movimientos básicos y acciones específicas.



Idea clave

Usa capas para que varias animaciones funcionen juntas sin interferirse, logrando movimientos más complejos y realistas.

Ejemplo: Capas para un personaje



Capa superior (acciones)

Ataque, salto, usar objeto



+

Capa base (locomoción)

Idle, Walk, Run



Resultado

La acción de ataque se superpone al movimiento base del personaje.

Ventana Animator: capas



Las capas se evalúan de abajo (Base Layer) hacia arriba (capas superiores).

Ejemplo práctico: capa de ataque (Upper Body)



Configuración de capas

Propiedad	Descripción	Recomendación
Weight (Peso)	Define la influencia de la capa (0 = sin influencia, 1 = influencia total).	Usa 1 para capas de acciones.
Blending	Modo de combinación entre capas.	Override: la capa superior reemplaza huesos definidos.
Avatar Mask	Determina qué partes del cuerpo afecta la capa.	Asigna máscaras para aislar partes del cuerpo (ej.: solo brazos).
IK Pass	Permite aplicar IK después de mezclar las capas.	Útil para ajustar pies o manos.

Buenas prácticas

- ✓ Usa la Base Layer solo para locomoción.
- ✓ Crea capas adicionales para acciones específicas.
- ✓ Emplea Avatar Masks para evitar afectar partes innecesarias.
- ✓ Ajusta el Weight para transiciones suaves entre capas.
- ✓ Organiza y nombra bien tus capas para mantener claridad.



Tip pro

Para ataques, usa una capa superior con máscara de brazos y torso. Así puedes atacar sin detener el movimiento de las piernas.



En resumen

Las capas en Animator permiten superponer animaciones de manera controlada. Úsalas para separar locomoción y acciones, logrando personajes más dinámicos y realistas.



Recuerda: Las capas se evalúan de abajo hacia arriba. La capa superior tiene prioridad sobre la inferior.





¿Qué son los estados?

Los estados representan los diferentes clips de animación que puede reproducir un personaje.

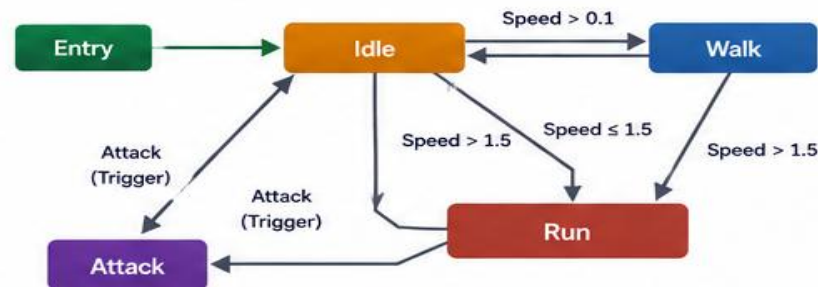
La máquina de estados permite cambiar de un estado a otro según condiciones específicas.



Idea clave

Usa estados y transiciones para crear comportamientos dinámicos y responder a las acciones del jugador o condiciones del juego.

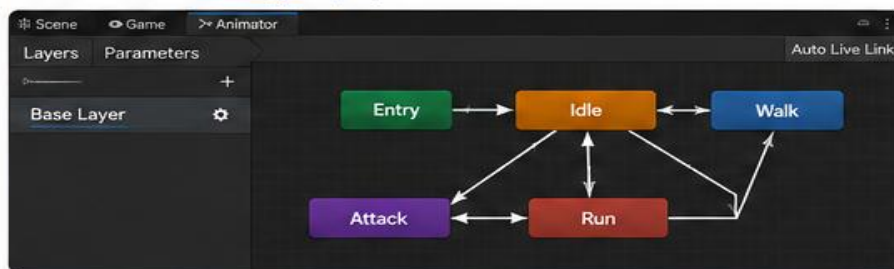
Ejemplo: máquina de estados básica



¿Cómo funciona?

- 1 El Entry es el punto de entrada.
- 2 Cada rectángulo es un estado (clip de animación).
- 3 Las flechas son transiciones entre estados.
- 4 Las condiciones (parámetros) determinan cuándo ocurre la transición.

Ventana Animator (ejemplo)



Organiza los estados y transiciones en la ventana Animator para definir el flujo de animación.

Tipos de transiciones



Transición bidireccional

Permite ir y volver entre dos estados.



Transición unidireccional

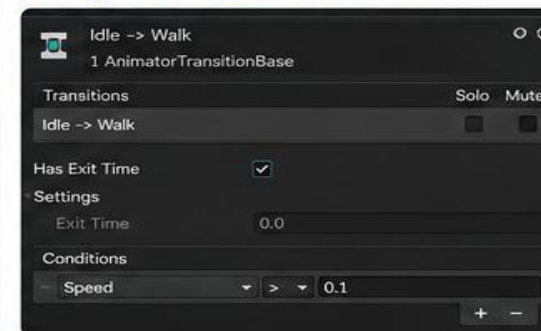
Solo permite ir de un estado a otro.



Transición con interrupción

Puede interrumpir el estado actual para ir a otro (ej.: ataque).

Configuración de una transición



Elementos clave

- Has Exit Time: espera a un punto específico del clip actual.
- Conditions: define las condiciones para realizar la transición.
- Solo: evita que otras transiciones se activen al mismo tiempo.
- Mute: desactiva temporalmente la transición.

Parámetros usados en transiciones

Tipo	Descripción	Ejemplo de uso
Float	Valor decimal (puede ser negativo).	Speed, Direction
Int	Número entero.	Combo, Ammo
Bool	Verdadero o falso.	IsGrounded, IsRunning
Trigger	Se activa una vez y se reinicia.	Attack, Jump
Vector	Valor con 2, 3 o 4 componentes.	Move (X, Y, Z)

Ejemplo práctico: transición por velocidad



Buenas prácticas

- ✓ Usa nombres claros para estados y parámetros.
- ✓ Evita transiciones contradictorias.
- ✓ Ajusta los tiempos de salida (Exit Time) para transiciones suaves.
- ✓ Prueba todas las transiciones en Play Mode.
- ✓ Usa capas para separar acciones superiores (ej.: ataque) de locomoción.



Consejo: Menos es más. Diseña una máquina de estados simple y escalable.



En resumen

Los estados y transiciones permiten controlar el flujo de animaciones de forma inteligente. Combinados con parámetros, crean comportamientos adaptables y responsivos.



Recuerda: Una buena máquina de estados hace que tus personajes se sientan vivos y creíbles.





¿Qué son los parámetros?

Son variables que controlan las transiciones entre estados. Pueden ser de diferentes tipos y se combinan con condiciones para crear comportamientos precisos.



Idea clave

Combina parámetros y condiciones para que tu personaje responda de forma realista a las acciones del jugador y del entorno.

Tipos de parámetros en Unity

Tipo	Descripción	Ejemplo de uso
Float	Valor decimal (puede ser negativo).	Velocidad, dirección, tiempo
Int	Número entero.	Salud, munición, puntaje
Bool	Verdadero o falso.	Está en el suelo, atacando
Trigger	Se activa una vez y se reinicia.	Atacar, saltar, recibir daño



Los triggers son ideales para acciones puntuales que no deben mantenerse activas.

Ejemplo: parámetros comunes



Speed (Float)
Controla la velocidad del personaje.



IsGrounded (Bool)
Indica si el personaje está en el suelo.



Attack (Trigger)
Dispara la animación de ataque.



Health (Int)
Controla la cantidad de vida del jugador.

Combinando condiciones

Puedes usar AND (&&) y OR (||) para crear transiciones más precisas.

AND (todas deben cumplirse)



Ejemplo: `Speed > 0.1 && IsGrounded == true`
El personaje camina solo si se mueve y está en el suelo.

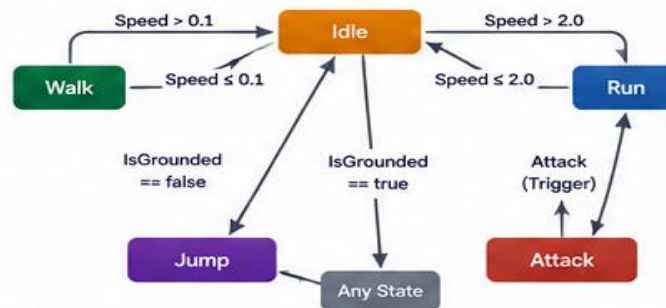
OR (cualquiera puede cumplirse)



Ejemplo: `Speed > 0.1 || IsGrounded == false`
El personaje corre si se mueve o si está en el aire.

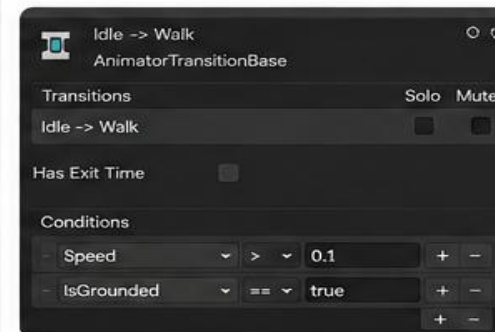
Puedes combinar varias condiciones para obtener resultados muy específicos.

Ejemplo práctico: transiciones con condiciones



Any State permite que un trigger (como Attack) interrumpa cualquier estado.

Inspector: condiciones de transición



Explicación

- ✓ **Speed > 0.1:** el personaje se está moviendo.
 - ✓ **IsGrounded == true:** el personaje está en el suelo.
- Solo si ambas condiciones se cumplen, se realiza la transición.

Uso de Has Exit Time

Permite que la transición ocurra después de que el estado actual haya alcanzado un porcentaje específico de su duración.

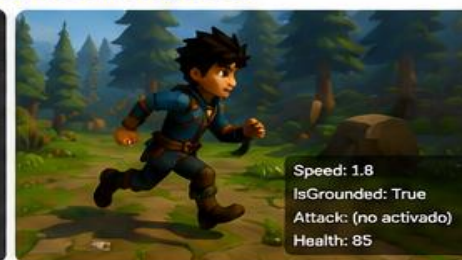


La transición solo puede ocurrir después del 75% de la animación actual.

Consejos para parámetros

- ✓ Usa nombres descriptivos y consistentes.
- ✓ Evita tener demasiados parámetros innecesarios.
- ✓ Actualiza los parámetros desde el código solo cuando sea necesario.
- ✓ Usa rangos lógicos para los valores (ejemplo: Speed de 0 a 5).
- ✓ Prueba en Play Mode y observa los cambios en tiempo real.

Visualización de parámetros en tiempo real



Errores comunes

- ✗ No actualizar los parámetros desde el código.
- ✗ Usar Has Exit Time cuando no es necesario.
- ✗ Crear transiciones sin condiciones.
- ✗ Valores de parámetros fuera de rango lógico.
- ✗ No probar las transiciones en Play Mode.



En resumen

Los parámetros y condiciones son la base para crear animaciones inteligentes. Úsalos correctamente para que tu personaje reaccione de forma natural y realista.



Recuerda: La clave está en combinar los parámetros adecuados con condiciones lógicas y probar constantemente.





¿Por qué optimizar?

Una máquina de estados bien optimizada mejora el rendimiento del juego y facilita su mantenimiento.

Algunas buenas prácticas ayudan a mantener tus animaciones claras, eficientes y escalables.



Idea clave

Organiza tus estados, usa transiciones claras, evita transiciones innecesarias y prueba constantemente en Play Mode.

Buenas prácticas recomendadas

- ✓ Usa nombres descriptivos para estados y parámetros.
- ✓ Evita transiciones ambiguas o contradictorias.
- ✓ Utiliza condiciones simples y lógicas.
- ✓ Ajusta los tiempos de transición para que sean suaves.
- ✓ Agrupa estados relacionados (Sub-State Machines).
- ✓ Prueba frecuentemente en Play Mode.
- ✓ Documenta tu Animator (comentarios y orden visual).

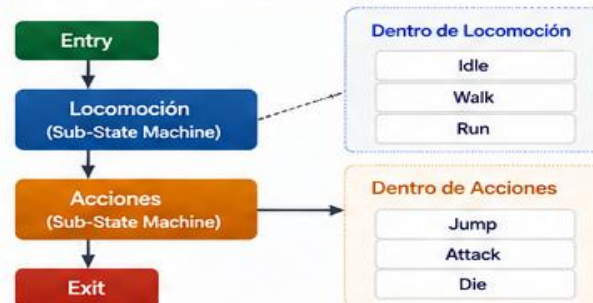
Evita estos errores comunes

- ✗ Demasiadas transiciones desde un estado.
- ✗ Transiciones sin condiciones.
- ✗ Parámetros que no se usan.
- ✗ Valores fuera de rango lógico.
- ✗ No probar las animaciones en Play Mode.

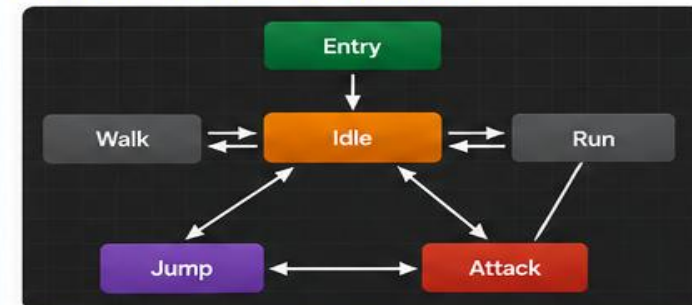
Sub-State Machines (máquinas de estados anidadas)

Permiten organizar grupos de estados complejos dentro de una máquina principal. Ideal para proyectos grandes.

i Ejemplo: separar locomoción de acciones.



Ejemplo: estructura optimizada

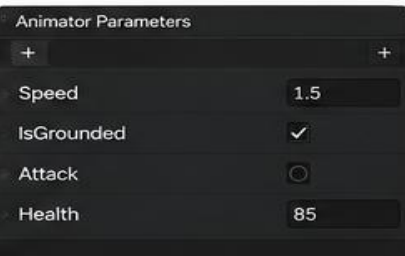
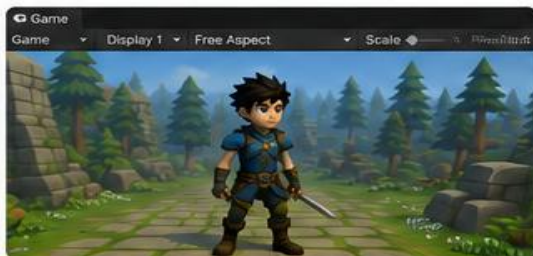


i Estados organizados, transiciones claras y lógicas.

Consejos de rendimiento

- Desactiva animaciones que no están en uso.
- Usa capas (Layers) solo cuando sea necesario.
- Evita muchas transiciones y condiciones complejas.
- Reduce el número de parámetros innecesarios.
- Usa Blend Trees para movimientos suaves.

Probando y depurando en Play Mode



i Ajusta los parámetros en tiempo real para ver los cambios en tus animaciones.

Checklist final antes de publicar

- ✓ Todos los estados tienen nombres claros.
- ✓ Las transiciones tienen condiciones lógicas.
- ✓ No hay transiciones innecesarias.
- ✓ Los parámetros tienen valores correctos.
- ✓ Las animaciones se ven suaves en Play Mode.
- ✓ El rendimiento es estable.
- ✓ El Animator está bien organizado y documentado.



Tip pro

Mantén tu Animator simple y modular. Si un comportamiento se vuelve muy complejo, divídelo en Sub-State Machines o controladores separados. Esto facilita el mantenimiento y la colaboración en equipo.



En resumen

Optimizar tu Animator es clave para crear animaciones fluidas, eficientes y fáciles de mantener. Organiza, prueba y mejora constantemente.





¿Por qué depurar?

La depuración te permite identificar problemas en los estados, transiciones, parámetros y condiciones para asegurarte de que las animaciones funcionen como esperas.



Idea clave

Usa las herramientas de Unity y Animator para probar, identificar errores y ajustar tu máquina de estados hasta que esté lista para el juego.

Herramientas de depuración en Animator



Solo (Solo): Aísla un estado o transición para probarlo.



Mute (Silenciar): Desactiva temporalmente una transición.



Parametros: Observa y modifica los valores en tiempo real.

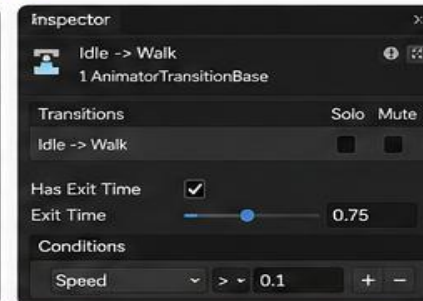
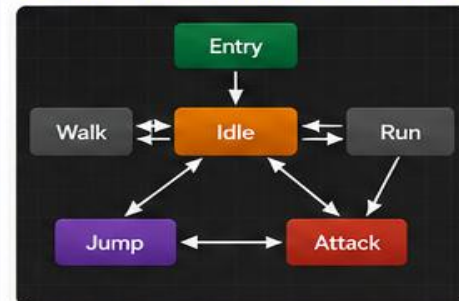


Play Mode: Prueba las animaciones en tiempo real.



Console: Revisa mensajes y errores del sistema.

Ejemplo: depuración de una transición



Usa Solo para probar la transición y ajusta las condiciones si no se activa correctamente.

Flujo de depuración recomendado



1. Identificar

Observa el comportamiento incorrecto en Play Mode.



2. Aislar

Usa Solo/Mute para aislar el estado o transición problemática.



3. Analizar

Revisa parámetros, condiciones y tiempos de salida (Exit Time).



4. Ajustar

Corrige valores, condiciones o transiciones.



5. Probar

Vuelve a Play Mode y verifica que el problema se haya solucionado.

Visualización de parámetros en tiempo real



Modifica los valores de los parámetros y observa cómo cambia la animación.

Errores comunes y soluciones

Error común	Causa probable	Solución
La transición no se activa.	Condición incorrecta o parámetro no cambia.	Verifica el nombre del parámetro y el rango del valor.
La animación se corta antes.	Exit Time muy bajo o sin transición de salida.	Ajusta el Exit Time o añade transiciones de salida.
El personaje flota o se hunde.	IsGrounded no funciona correctamente.	Verifica el raycast o el collider de suelo.
Se traba en un estado.	No hay transición de salida válida.	Añade transiciones hacia otros estados o Any State.

Ejemplo práctico: prueba en Play Mode

1. Problema inicial

El personaje no ataca cuando se presiona la tecla.



2. Depuración

Se revisa la condición del estado Attack (Trigger).



3. Solución

Se activa el Trigger correctamente y la animación funciona.



Probar, depurar y ajustar es clave para obtener animaciones fluidas y profesionales.

Checklist de depuración antes de publicar

- ✓ Todos los estados tienen nombre descriptivo.
- ✓ Las transiciones tienen condiciones correctas.
- ✓ Los parámetros tienen valores dentro del rango lógico.
- ✓ No hay transiciones innecesarias o duplicadas.
- ✓ El Exit Time está configurado adecuadamente.
- ✓ Las animaciones cambian correctamente en Play Mode.
- ✓ No aparecen errores en la consola.
- ✓ La máquina de estados está organizada y documentada.



Tip pro

Duplica tu Animator Controller antes de hacer cambios grandes para no perder tu versión estable.

Documenta tu máquina de estados (diagrama + descripción) para facilitar futuras mejoras.



En resumen

Depurar y probar en Play Mode te permite detectar errores, ajustar parámetros y crear animaciones de alta calidad. La práctica constante mejorará tus habilidades como diseñador de videojuegos.





¿Qué son los Blend Trees?

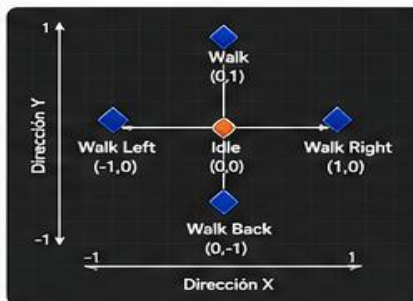
Permiten mezclar varias animaciones según uno o más parámetros, logrando transiciones suaves entre ellas, como variar la velocidad de movimiento o la dirección.



Idea clave

Usa Blend Trees para que las transiciones entre animaciones sean fluidas y el movimiento del personaje se vea natural y consistente.

Ejemplo: Blend Tree 2D para locomoción



Parámetros del Blend Tree 2D

- **Parámetro Horizontal (X):** -1 (izquierda) a 1 (derecha)
- **Parámetro Vertical (Y):** -1 (atrás) a 1 (adelante)



El Animator mezcla las animaciones según los valores actuales de los parámetros X e Y.

Visualización en el juego



Tipos de Blend Trees

Blend Tree 1D



Mezcla animaciones según un solo parámetro (ej.: velocidad).

Blend Tree 2D



Mezcla animaciones según dos parámetros (eje X e Y) (ej.: dirección).



Los Blend Trees evitan transiciones bruscas y mejoran el realismo del movimiento.

Ejemplo práctico: Blend Tree 1D por velocidad

Parámetro: Speed (0 = Idle, 1 = Run)

0 ————— 0.5 ————— 1

Idle

Walk

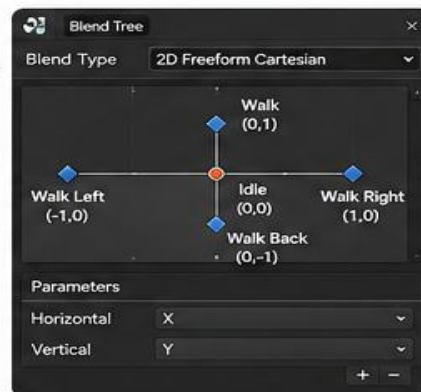
Run



Resultado: El personaje cambia suavemente de Idle a Run según el valor de Speed.

Crear un Blend Tree en Unity

- 1 Clic derecho en el Project > Create > Animator Controller.
- 2 Abre el Animator y crea un nuevo estado.
- 3 Clic derecho en sii estado > Make Transition > Blend Tree.
- 4 Selecciona 1D o 2D según tus necesidades.
- 5 Agrega las animaciones y establece los parámetros.
- 6 Guarda y prueba en Play Mode.



Beneficios de usar Blend Trees

- ✓ Movimientos más fluidos y naturales.
- ✓ Menos transiciones y condiciones complejas.
- ✓ Mejor rendimiento y mantenimiento.
- ✓ Ideal para locomoción y acciones combinadas.
- ✓ Compatibles con múltiples capas.

Buenas prácticas

- ✓ Define rangos claros para tus parámetros.
- ✓ Evita valores fuera de rango (-1 a 1).
- ✓ Prueba siempre en Play Mode.
- ✓ Combina con capas para acciones superiores.
- ✓ Documenta tus parámetros y valores usados.

Parámetros comunes para Blend Trees

Parámetro	Tipo	Descripción	Ejemplo de uso
Speed	Float	Controla la velocidad de movimiento.	0 (quieto) a 1 (correr)
Direction X	Float	Controla la dirección horizontal.	-1 (izq.) a 1 (der.)
Direction Y	Float	Controla la dirección vertical.	-1 (atrás) a 1 (adelante)
Blend	Float	Controla mezclas personalizadas.	Transiciones entre acciones específicas.



Tip pro

Usa nombres claros para tus parámetros y anima con valores normalizados (-1 a 1) para mejores resultados.

Errores comunes

- ✗ Valores de parámetros fuera de rango.
- ✗ Transiciones innecesarias entre animaciones.
- ✗ No probar en Play Mode.
- ✗ No usar capas para acciones superiores.
- ✗ No documentar parámetros y configuraciones.



Revisa tus parámetros y pruebas constantemente para obtener movimientos naturales y precisos.



En resumen

Los Blend Trees son esenciales para crear movimientos fluidos y realistas. Úsalos para mezclar animaciones según parámetros y mejora la experiencia del jugador.



Recuerda: Menos transiciones = más fluidez. Prueba, ajusta y documenta.





¿Qué son los clips de animación?

Un clip de animación almacena los movimientos de un personaje.

Puedes reutilizarlos en diferentes transiciones y estados para ahorrar tiempo y mantener consistencia.

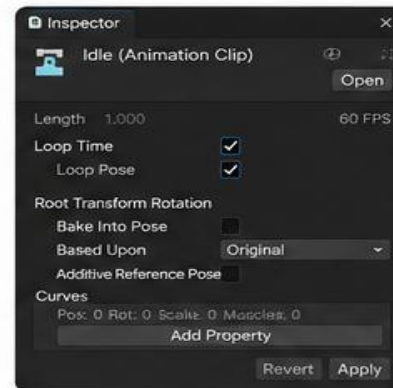


Idea clave

Organiza tus clips de animación, ajusta sus propiedades y controla la velocidad para crear movimientos fluidos y efectivos.

Importar y gestionar clips

- 1 Selecciona el archivo de animación (FBX).
- 2 Ve a la pestaña "Animation" en el Inspector.
- 3 Configura las opciones de importación.
- 4 Haz clic en "Apply".
- 5 El clip estará disponible en el panel de Project.



Ejemplo: clip Idle



0.00 s

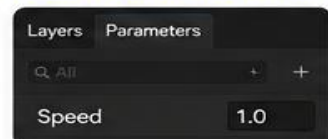
0.50 s

1.00 s

i El clip Idle se reproduce en bucle para mantener al personaje siempre listo.

Control de velocidad de animación

Parámetro Speed en el Animator



i El parámetro Speed multiplica la velocidad base del clip de animación.

Ejemplo de control de velocidad



Organización de clips en el Project

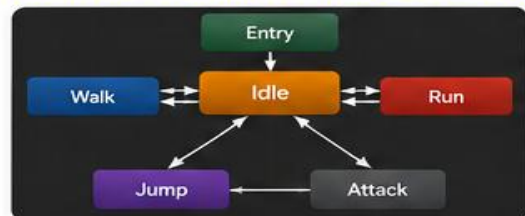


i Mantén tus clips organizados para facilitar su uso y reutilización.

Propiedades importantes de los clips

Propiedad	Descripción	Recomendación
Loop Time	Reproduce el clip en un bucle continuo.	Úsalo en Idle, Walk, Run.
Loop Pose	Mantiene la pose inicial al hacer loop.	Activado para transiciones suaves.
Root Transform Rotation	Controla la rotación del objeto raíz.	Basado en "Original" o "Bake Into Pose".
Additive	Permite mezclar el clip sobre otros.	Úsalo para capas adicionales (ej: gestos).
Speed	Controla la velocidad de reproducción.	Ajusta con parámetro Speed del Animator.

Reutilización de clips en múltiples estados



- ✓ Un mismo clip puede usarse en diferentes transiciones.
- ✓ Esto ahorra espacio y mantiene coherencia visual.
- ✓ Facilita ajustes globales al modificar el clip original.

Consejos para trabajar con clips

- ✓ Usa nombres descriptivos para cada clip.
- ✓ Evita clips demasiado largos; divide si es necesario.
- ✓ Ajusta la duración de los clips para que sean naturales.
- ✓ Prueba tus clips en diferentes velocidades.
- ✓ Utiliza curvas de animación para suavizar movimientos.
- ✓ Combina clips con capas para mayor detalle.



Problemas comunes

- ✗ Clips sin Loop Time en Idle.
- ✗ Transiciones bruscas por clips mal ajustados.
- ✗ Velocidades fuera de rango.
- ✗ Clips demasiado largos o con pausas innecesarias.
- ✗ Animaciones que no coinciden en altura o posición.

Solución

Ajusta las propiedades del clip, prueba en Play Mode y realiza transiciones suaves con condiciones adecuadas.



En resumen

Los clips de animación te permiten reutilizar movimientos y controlar su velocidad. Una buena gestión de clips mejora la calidad, el rendimiento y la consistencia de tus animaciones.



Recuerda: Organiza tus clips, ajusta las velocidades y prueba siempre en Play Mode para obtener resultados profesionales.





¿Qué son las curvas de animación?

Las curvas de animación controlan cómo cambia el valor de una propiedad en el tiempo. Con ellas puedes crear movimientos más naturales, evitando cambios bruscos o lineales.



Idea clave

Usa curvas para suavizar el inicio y el final de una animación, y para dar sensación de peso, inercia o aceleración.

Tipos de curvas comunes



Lineal
Velocidad constante.



Ease In
Comienza lento y acelera.



Ease Out
Comienza rápido y termina lento.



Ease In Out
Comienza lento, acelera y vuelve a frenar.



Bounce
Rebota al final de la animación.

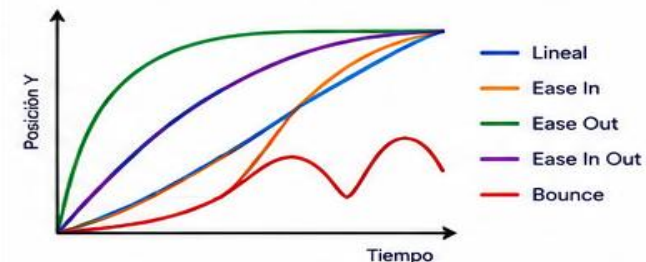


Elastic
Efecto elástico con rebote.



Las curvas ayudan a que las animaciones se vean más orgánicas y agradables.

Ejemplo visual de curvas (posición Y)



Curvas predefinidas en Unity (Animation Window)



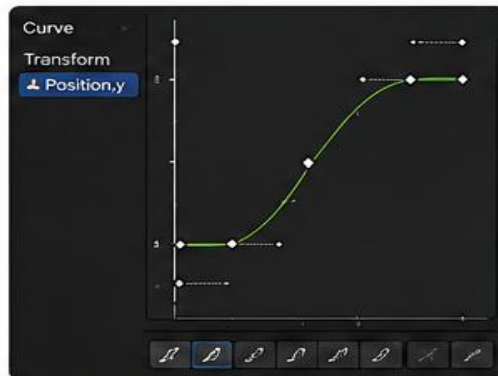
Unity incluye muchas curvas predefinidas que puedes aplicar con un clic.

- ✓ Ease In / Ease Out
- ✓ Ease In Out
- ✓ Bounce In / Out
- ✓ Elastic In / Out
- ✓ Curve personalizada



Puedes editar los keyframes y ajustar la curva arrastrando los manejadores.

Editar curvas personalizadas



Consejos

- ✓ Arrastra los manejadores para cambiar la forma de la curva.
- ✓ Evita curvas muy bruscas o con cambios repentinos.
- ✓ Prueba siempre en Play Mode.

Ejemplo práctico: salto con curvas



Combinar curvas crea movimientos más realistas y expresivos.

Aplicaciones de las curvas



Salto y caídas: suaviza el impulso y la gravedad.



Cámaras: movimientos suaves de seguimiento y acercamientos.



Efectos UI: apariciones, desvanecimientos y transiciones agradables.



Objetos y físicas: sensación de peso e inercia.

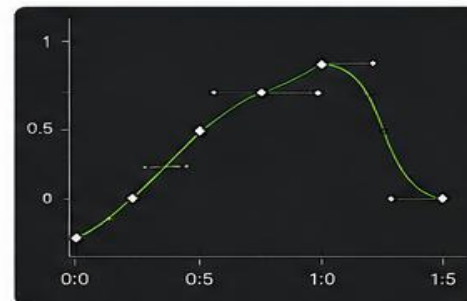
Buenas prácticas

- ✓ Usa Ease In Out para la mayoría de los movimientos.
- ✓ No abuses de curvas complejas.
- ✓ Ajusta la duración junto con la curva.
- ✓ Observa y prueba en Play Mode.
- ✓ Mantén consistencia en el estilo de movimiento del juego.

Errores comunes

- ✗ Usar solo curvas lineales: movimientos mecánicos.
- ✗ Cambios bruscos en la curva.
- ✗ Duraciones muy cortas para curvas complejas.
- ✗ Rebotes innecesarios o exagerados.
- ✗ No probar la animación en el contexto real del juego.

Ejemplo de curva personalizada



Tip pro

Observa animaciones del mundo real (deportes, naturaleza, personas) y trata de replicar sus curvas. La referencia es clave para lograr movimientos creíbles.



En resumen

Las curvas de animación permiten controlar la velocidad y suavizar los movimientos. Úsalas para dar realismo, mejorar la experiencia y destacar la calidad de tu juego.





¿Qué aprendimos?

Las curvas de animación permiten controlar la velocidad y suavizar los movimientos, haciendo que nuestras animaciones se vean más naturales, expresivas y profesionales.

Aprendimos a usar tipos de curvas, editarlas, combinarlas con clips y a gestionar la velocidad de reproducción.



Ideas clave de la sesión

- ✓ Las curvas controlan la velocidad a lo largo del tiempo.
- ✓ Existen varios tipos de curvas para diferentes necesidades.
- ✓ Puedes crear y editar curvas personalizadas.
- ✓ Los clips permiten reutilizar y organizar animaciones.
- ✓ El control de velocidad mejora la fluidez y coherencia de tus animaciones.

Tipos de curvas comunes



Lineal

Velocidad constante.



Ease In

Comienza lento y acelera.



Ease Out

Comienza rápido y termina lento.



Ease In Out

Comienza lento, acelera y vuelve a frenar.



Bounce

Rebota al final de la animación.



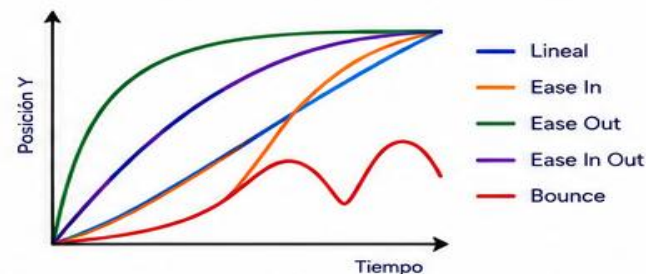
Elastic

Efecto elástico con rebote.



Elige la curva adecuada para lograr el estilo de movimiento que necesitas.

Ejemplo visual de curvas (posición Y)



Flujo completo trabajado



Importar y gestionar clips

Importa tus clips de animación y organízalos.



Editar curvas

Modifica las curvas para suavizar y personalizar.



Controlar velocidad

Ajusta el parámetro Speed para controlar la velocidad base.



Combinar y probar

Combina curvas, prueba en Play Mode y ajusta.



Resultado

Animaciones más naturales, fluidas y profesionales.



Itera, observa y mejora constantemente tus animaciones.

Consejos para seguir mejorando

- ✓ Observa animaciones del mundo real y trata de replicarlas.
- ✓ Usa curvas personalizadas para darle tu sello único.
- ✓ Combina curvas para movimientos más expresivos.
- ✓ Ajusta siempre en el contexto real del juego.
- ✓ No sobrecargues las curvas con muchos keyframes innecesarios.
- ✓ Documenta tus configuraciones para reutilizarlas.



La práctica constante es la clave para dominar las animaciones y crear experiencias increíbles.

Buenas prácticas



Usa nombres claros para tus clips y curvas.



Organiza tus clips en carpetas y subcarpetas.



Ajusta la duración del clip según la acción que representa.



Prueba siempre en Play Mode.



Combina curvas y parámetros para mayor realismo.



Mantén consistencia en el estilo de movimiento del juego.

Errores comunes

- ✗ Usar solo curvas lineales.
- ✗ Cambios bruscos en la curva.
- ✗ Duraciones muy cortas para curvas complejas.
- ✗ Rebotes innecesarios o exagerados.
- ✗ Velocidades fuera de rango.
- ✗ Clips mal organizados o sin nombres claros.



Checklist final

- ✓ Mis clips están bien organizados.
- ✓ Mis curvas están ajustadas y probadas.
- ✓ Mis velocidades están configuradas.
- ✓ Probé todo en Play Mode.
- ✓ Mi animación se ve natural y fluida.
- ✓ Documenté mis configuraciones.



¿Qué sigue?

En las próximas sesiones aprenderemos sobre estados de animación, máquinas de estados y blending, para crear animaciones más avanzadas e interactivas.

¡Sigue explorando y creando!



En resumen

Las curvas de animación son una herramienta poderosa para dar vida y personalidad a tus personajes y objetos. Cuando aprendes a usarlas bien, tus juegos se sienten más profesionales y atractivos para los jugadores.

¡Tú tienes el control del movimiento!



¡Felicidades!

Has completado esta sesión.
¡Sigue creando mundos increíbles!





Referencias bibliográficas y recursos

i Consulta estas fuentes para profundizar tus conocimientos sobre animación y desarrollo en Unity.



Libros

- Burgess, J. (2020). *Unity 2019 Game Development Essentials*. Packt Publishing.
- Dichter, S. (2019). *The Ultimate Guide to Game Character Creation with Blender and Unity*. 3dtotal Publishing.
- Murray, D. (2018). *Unity in Action: Multiplatform Game Development in C#*. Manning Publications.
- Rickman, K. (2016). *Game Animation: Fundamentals and Controls*. CRC Press.
- Zacny, K., & Kerlow, A. (2010). *The Animator's Survival Kit: A Manual of Methods, Principles and Formulas for Classical, Computer, Games, Stop Motion and Internet Animators*. Faber & Faber.



Documentación oficial

- Unity Technologies. (s. f.). *Manual de Unity*. <https://docs.unity3d.com/Manual/index.html>
- Unity Technologies. (s. f.). *Scripting API*. <https://docs.unity3d.com/ScriptReference/>
- Unity Learn. (s. f.). *Pathways: Junior Programmer*. <https://learn.unity.com/>



Videos recomendados

- Brackeys. (2020). *Unity 2D Animation Tutorial for Beginners*. [Video]. YouTube. <https://www.youtube.com/c/Brackeys>
- Code Monkey. (2021). *Animator Controller en Unity*. [Video]. YouTube. <https://www.youtube.com/c/CodeMonkeyUnity>
- Unity. (2022). *Animation Fundamentals in Unity*. [Video]. YouTube. <https://www.youtube.com/@unity>



Recursos adicionales

Sitios web

- GameDev.net – Tutorials & Articles
<https://www.gamedev.net/>
- Gamasutra – Animation Techniques
<https://www.gamasutra.com/>
- Kenney – Game Dev Resources (assets y tutoriales)
<https://kenney.nl/>

Lecturas complementarias

- Ian Millington. (2019). *AI for Games* (3rd ed.). CRC Press.
- Mark J. P. Wolf. (2017). *Unity 2017 Game Development Essentials*. Packt Publishing.
- Richard Lemarchand. (2014). *Practical Maya Programming with Python*. Addison-Wesley.



Tip

Dedica tiempo a explorar la documentación oficial de Unity y practica constantemente para dominar las animaciones.



¡Felicitaciones!

Has completado esta sesión.
¡Sigue creando mundos increíbles!